UNIT-IV

Short Answer Type Questions

1. What is data analysis?

Data analysis is the process of **inspecting, cleaning, transforming, and modeling data** with the goal of discovering useful information, drawing conclusions, and supporting decision-making.

2. Name any two libraries in Python used for data visualization.

Two of the most popular Python libraries for data visualization are **Matplotlib** and **Seaborn**.

3. List two types of data visualization techniques.

Two common data visualization techniques are **bar charts**, used for comparing categories, and **scatter plots**, used for observing relationships between two numerical variables.

4. What function is used to load a CSV file in MATLAB?

The readtable() function is used in MATLAB to load data from a CSV file into a table variable.

Example: data = readtable('filename.csv');

5. Mention any two chart types supported by Seaborn.

Seaborn supports a wide variety of advanced statistical charts.

Bar Plot: Used to show comparisons between categories.

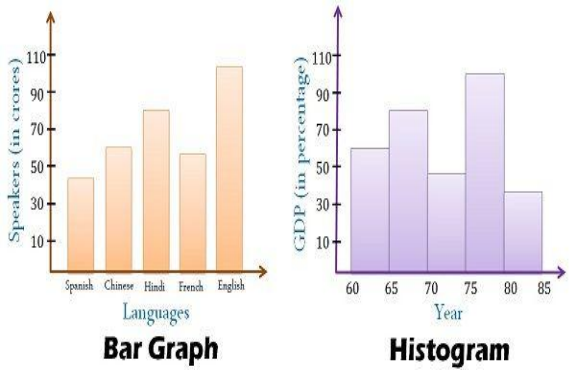
Example: sns.barplot(x='category', y='value', data=df)

Scatter Plot: Used to show the relationship between two numeric variables.

Example: sns.scatterplot(x='height', y='weight', data=df)

6. Differentiate between a bar chart and a histogram.

A **bar chart** compares data across distinct, separate categories (e.g., sales per city), with gaps between the bars. A **histogram** shows the frequency distribution of a single, continuous numerical variable (e.g., distribution of student heights) by grouping data into bins with no gaps between bars.



7. What is the role of groupby() in data analysis using Pandas?

The groupby() function is used to **split data into groups based on some criteria** (e.g., by category, date, or condition). After grouping, you can apply an aggregate function (like sum(), mean(), count()) to each group independently, which is a core part of data analysis.

8. Why is Seaborn preferred over Matplotlib in some cases?

Seaborn is often preferred because it provides a **higher-level interface for creating statistically sophisticated and aesthetically pleasing plots** with less code. It also integrates seamlessly with Pandas DataFrames, simplifying the plotting of complex data.

9. How do you filter rows in a Pandas DataFrame where the age is greater than 30?

You can filter rows using boolean indexing. Assuming the DataFrame is named df, the code is:

```
df[df['age'] > 30]
```

10. Which Python function is used to plot a pie chart?

The **pie()** function from the Matplotlib library, typically called as **matplotlib.pyplot.pie()** or **plt.pie(),** is used to create a pie chart.

11. Identify a suitable visualization technique to show correlation between two numeric variables.

A **scatter plot** is the most suitable technique to visualize the correlation and relationship between two numeric variables. A trend line can be added to further clarify the strength and direction of the correlation.

12. What are the key differences between a Pandas DataFrame and a MATLAB table for data manipulation?

The key difference lies in the ecosystem.

1. Language and Syntax:

Pandas DataFrame (Python) allows flexible data access using labels and positions, while MATLAB table mainly uses column names or positions to access data.

DATAFRAM EXAMPLE:

```
df.loc[0, 'Name']   # Access by label
df.iloc[0, 1]       # Access by position
```

MATLAB EXAMPLE:

```
T.Name(1)   % Access by column name
T{1, 2}     % Access by position
```

2. Data Handling:

Pandas handles missing data with functions like **dropna()** and **fillna().**
MATLAB uses **rmmissing()** and **fillmissing()** for similar operations.

13. Which platform—Python or MATLAB—would you choose for building real-time dashboards and why?

I would choose **Python**. Libraries like **Dash** and **Streamlit** are specifically designed for building interactive, web-based dashboards directly from Python data analysis scripts. They offer greater flexibility for web deployment and integration with other web technologies compared to MATLAB's more specialized dashboarding tools.

14. Evaluate the limitations of pie charts in data visualization.

Pie charts have several limitations:

Poor for Comparison: It's difficult for the human eye to accurately compare the sizes of different slices, especially if they are similar.

Information Density: They take up a lot of space to display a small amount of information.

Too Many Categories: They become cluttered and unreadable if there are more than a handful of categories. A bar chart is almost always a better alternative.

Long Answer Type Questions

1. Describe the key steps involved in the data analysis process.

Data Analysis is the process of inspecting, cleaning, transforming, and modeling data to discover useful information, draw conclusions, and support decision-making.

Problem Definition & Data Collection: First, clearly define the question you want to answer. Based on this, you gather relevant data from various sources like databases, APIs, or files (e.g., CSV, Excel).

Data Cleaning (Wrangling): This is often the most time-consuming step. It involves handling inconsistencies in the data, such as:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Empty Cells

Empty cells can potentially give you a wrong result when you analyze data.

One way to deal with empty cells is to remove rows that contain empty cells.

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna()
print(new_df.to_string())
```

Data of Wrong Format

Cells with data of wrong format can make it difficult, or even impossible, to analyze data. Converting columns to the correct format (e.g., text to numbers, strings to dates).

To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

```
import pandas as pd
df = pd.read_csv('data.csv')
df['Date'] = pd.to_datetime(df['Date'], format='mixed')
print(df.to_string())
```

Wrong Data

"Wrong data" does not have to be "empty cells" or "wrong format", it can just be wrong, like if someone registered "450" instead of "50".

One way to fix wrong values is to replace them with something else.

```
df.loc[7, 'Duration'] = 45
```

Discovering Duplicates

Duplicate rows are rows that have been registered more than one time.Identifying and removing redundant entries.

```
df.drop_duplicates(inplace = True)
```

Exploratory Data Analysis (EDA): In this step, you explore the data to understand its underlying structure and patterns. This involves:

Descriptive statistics: Calculating measures like mean, median, and standard deviation.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.describe())
```

Data visualization: Creating charts like histograms, scatter plots, and bar plots to visually identify trends, correlations, and anomalies.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
sns.histplot(data=df, x='Maxpulse', bins=5, kde=True, color='skyblue')
plt.title("Distribution of Maxpulse")
plt.xlabel("Maxpulse")
plt.ylabel("Frequency")
plt.show()
```

Modeling & Analysis: Based on the insights from EDA, you apply more formal techniques. This could range from simple grouping and aggregation to building complex machine learning models to make predictions or classify data.

```
import pandas as pd
df = pd.read_csv('exercise_data.csv')
daily_avg=df.groupby('Date')['Pulse'].mean()
print(daily_avg)
```

Interpretation & Communication: The final step is to interpret the results of your analysis and communicate the findings to stakeholders. This is often done by creating reports or dashboards with clear visualizations that tell a story and answer the initial question. The results should be actionable and lead to informed decisions.

2. Explain different types of basic data visualization techniques with examples.

Data visualization techniques are used to represent data graphically, making complex data more understandable. some basic types are:

Bar Chart: Used to compare numerical values across discrete categories. The length of each bar is proportional to the value it represents.

Example: Comparing the number of products sold in different countries.

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]
# Create bar chart
plt.bar(categories, values, color='skyblue')
plt.title("Bar Chart Example")
plt.xlabel("Category")
plt.ylabel("Values")
plt.show()
```

Line Chart: Used to display a trend over a continuous interval, most commonly time. It connects a series of data points with a line.

Example: Tracking a company's stock price over one year.

```
import matplotlib.pyplot as plt
# Simple Line Plot
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)  # line plot
plt.title("Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

Scatter Plot: Used to show the relationship between two numerical variables. Each point on the plot represents an observation with coordinates corresponding to the two variables. It's excellent for identifying correlations.

Example: Plotting a person's height versus their weight to see if they are related.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.plot(kind = 'scatter', x = 'Duration', y = 'Calories')
plt.show()
```

Histogram: Used to visualize the distribution of a single continuous variable. It groups numbers into ranges (bins) and shows the frequency of data points falling into each bin.

Example: Visualizing the distribution of exam scores for a class of students.

```
import numpy as np
import matplotlib.pyplot as plt
measurements = np.random.normal(loc=10.0, scale=0.1, size=1000)
plt.hist(measurements, bins=30, edgecolor='black') # bins controls how many bars
plt.title("Distribution of Component Length Measurements")
plt.xlabel("Length (cm)")
plt.ylabel("Frequency")
plt.show()
```

Pie Chart: A circular chart divided into slices to illustrate numerical proportion. Each slice's angle is proportional to the quantity it represents, showing parts of a whole.

Example: Displaying the market share percentage for different smartphone brands.

```
import matplotlib.pyplot as plt
labels = ['Apples', 'Bananas', 'Oranges', 'Grapes']
sizes = [30, 25, 25, 20]
colors = ['red', 'yellow', 'pink', 'brown']
# Create pie chart
explode=(0,0.1,0,0)
plt.pie(sizes,explode=explode, labels=labels, colors=colors, autopct='%1.1f%%',
shadow=True,startangle=140)
plt.title("Fruits Distribution")
```

```
plt.axis('equal')  # Equal aspect ratio ensures the pie is a circle.
plt.show()
```

3. Compare the visualization features of Python (Matplotlib/Seaborn) and MATLAB.

Python and MATLAB are both powerful platforms for data visualization, but they cater to different needs and workflows.

Feature	Python (Matplotlib & Seaborn)	MATLAB
Core Philosophy	Open-source and part of a general-purpose language ecosystem.	Proprietary and part of a highly integrated numerical computing environment.
Primary Libraries	**Matplotlib:** Low-level, highly customizable for full control. **Seaborn:** High-level, built on Matplotlib for statistical plots.	Built-in plotting functions are core to the language.
Ease of Use	**Seaborn** is very easy for standard statistical plots. **Matplotlib** requires more code for customization.	Generally consistent and straightforward syntax for standard plots. Excellent interactive tools.
Integration	Excellent integration with Pandas DataFrames and the entire Python data science stack (Scikit-learn, etc.).	Tightly integrated with its own toolboxes (e.g., Signal Processing, Image Processing).
Strengths	Statistical visualization, integration with web applications (Dash), machine learning pipelines.	Engineering plots (3D, signals), mathematical visualization, interactive plot editing.

Feature	Python (Matplotlib & Seaborn)	MATLAB
Customization	**Extremely high** with Matplotlib, allowing for creation of virtually any plot imaginable.	High, but sometimes less flexible for highly non-standard or bespoke visualizations.
Aesthetics	**Seaborn** produces beautiful, modern-looking plots by default. Matplotlib's defaults are more basic.	Professional and publication-quality, but can look more traditional by default.

Conclusion:

Choose **Python** for its flexibility, powerful statistical plotting with Seaborn, and seamless integration into larger data science and web application workflows. Choose **MATLAB** for its superior interactive environment and specialized toolboxes, making it ideal for engineering, academia, and complex mathematical visualization.

4. Explain the importance of data analysis in Python and MATLAB with suitable use cases.

Data analysis is crucial for transforming raw data into actionable intelligence, and both Python and MATLAB are premier platforms for this, albeit with different areas of focus.

Importance in Python:

Python has become the de facto standard for data science in the business and tech industries due to its versatility, readability, and an unparalleled ecosystem of open-source libraries.

Pandas: For high-performance data manipulation and analysis.

NumPy: For fundamental scientific computing.

Scikit-learn: For machine learning.

Matplotlib/Seaborn: For visualization.

Use Cases for Python:

Business Analytics: A retail company uses Pandas to analyze sales data, groupby() to aggregate sales by region and product category, and Seaborn to visualize trends to optimize inventory.

Financial Modeling: Hedge funds use Python to analyze stock market data, build predictive models with Scikit-learn to forecast price movements, and backtest trading strategies.

Web Technology: A tech company analyzes user behavior data (e.g., clicks, time on page) to perform A/B testing, helping them decide which website layout leads to higher user engagement.

Importance in MATLAB:

MATLAB's importance lies in its powerful numerical computation engine and specialized toolboxes, making it indispensable in engineering, research, and scientific fields. Its integrated environment allows for rapid prototyping and analysis of complex mathematical systems.

Use Cases for MATLAB:

Signal Processing: An automotive engineer uses the Signal Processing Toolbox to analyze sensor data from a car's engine, applying filters to remove noise and using FFT (Fast Fourier Transform) to identify vibration frequencies, which can indicate mechanical problems.

Image and Video Processing: Medical researchers use the Image Processing Toolbox to analyze MRI scans, segmenting images to measure tumor size and track its changes over time in response to treatment.

Control Systems Design: Aerospace engineers model the flight dynamics of a drone in Simulink (a MATLAB-based environment), analyzing its stability and designing control systems to ensure it flies correctly under different conditions.

5. Demonstrate with code how to merge two dataframes in Pandas using a common key.

```
import pandas as pd
# Create the first DataFrame: student information
student_data = {
    'student_id': [101, 102, 103, 104],
    'name': ['Ali', 'Bilal', 'Cris', 'Dev'],
    'major': ['Physics', 'Chemistry', 'Math', 'Physics']
 }
students_df = pd.DataFrame(student_data)
# Create the second DataFrame: course enrollment
enrollment_data = {
    'student_id': [101, 102, 103, 105],
    'course': ['CSE', 'ECE', 'EEE', 'CIVIL']
 }
enrollment_df = pd.DataFrame(enrollment_data)
print(students_df)#print Students DataFrame
print(enrollment_df) #print Enrollment DataFrame
# Merge the two DataFrames using the 'student_id' column as the common key.
# An 'inner' merge is the default, which only keeps rows where the key exists in
BOTH DataFrames.
merged_df = pd.merge(students_df, enrollment_df, on='student_id')
print(merged_df) #print Merged DataFrame (Inner Join)

OUTPUT:

"C:\Users\masrath parveen\PycharmProjects\div3\.venv1\merge.py"
   student_id   name      major
0         101    Ali    Physics
1         102  Bilal  Chemistry
2         103   Cris       Math
3         104    Dev    Physics
```

```
  student_id course
0        101    CSE
1        102    ECE
2        103    EEE
3        105  CIVIL
  student_id   name   major    course
0        101    Ali    Physics    CSE
1        102  Bilal  Chemistry    ECE
2        103   Cris       Math    EEE
```

The above program clearly shows how pd.merge combines the data for students 101, 102, and 103 who exist in both tables

6. Analyze how groupby() and split-apply-combine work differently in Pandas and MATLAB with suitable examples.

The "split-apply-combine" strategy is a powerful pattern for data analysis. Both Pandas and MATLAB support it, but with different syntax and approaches.

Pandas (groupby)

In Pandas, this pattern is implemented using the **.groupby()** method on a DataFrame, followed by an aggregation method like **.sum(), .mean(), or .agg().**

In Pandas (Python): groupby() is directly available.

It **splits** the data based on a key (like a column), **applies** a function (like sum, mean), and **combines** the results.

.

Example in Python:

```
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
    'Team': ['A', 'A', 'B', 'B', 'C'],
    'Score': [10, 20, 15, 25, 30]
})
# Group by 'Team' and find average score
result = df.groupby('Team')['Score'].mean()
print(result)
OUTPUT:
Team
A    15.0
B    20.0
C    30.0
Name: Score, dtype: float64
```

This follows the split (by Team) → apply (mean) → combine (result) pattern.

MATLAB (findgroups and splitapply)

In MATLAB, the same pattern is achieved using two separate functions: **findgroups** to create numeric group identifiers and **splitapply** to apply a function to the data splits. The approach is more functional and procedural.

Example in MATLAB:

```
Team = {'A'; 'A'; 'B'; 'B'; 'C'};
Score = [10; 20; 15; 25; 30];
[G, groupNames] = findgroups(Team);
result = splitapply(@mean, Score, G)
OUTPUT
result =
   15
   20
   30
```

Here, findgroups does the splitting, splitapply applies the function, and MATLAB returns the result.

Both Pandas and MATLAB follow the Split-Apply-Combine pattern but differ in implementation. Pandas is simpler and more flexible, allowing one-liners using groupby(), while MATLAB requires two functions (findgroups, splitapply) to achieve the same result.

7. Examine the effect of different parameters (color, marker, linestyle) in enhancing the clarity of a scatter plot.

Parameters like color, marker, and size are crucial for enhancing a scatter plot because they allow you to encode additional dimensions of data onto a 2D plot, making it far more informative and clear. While linestyle is primarily for line plots, alpha (transparency) is a key parameter for scatter plots.

Using parameters like color, marker, and linestyle makes scatter plots much clearer by helping you visually separate different groups of data.

Why They Matter ?

Color: The easiest way to show which data points belong to which group.

Marker: Using different shapes (like circles o or squares s) adds another visual clue, which is great for accessibility (color-blindness) or when colors are too similar.

Linestyle: Connecting points with lines (like solid - or dashed ---) can show a sequence, trend, or relationship between them.

Let's examine their effect with a dataset:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Sample exercise data
data = {
    'Pulse': [110, 117, 103, 109, 98, 100, 120],
    'Calories': [409, 479, 340, 282, 406, 300, 500],
    'Duration': [60, 45, 45, 30, 60, 60, 45]
}
df = pd.DataFrame(data)
# Simple scatter plot with color and marker for clarity
sns.scatterplot(x='Pulse', y='Calories', data=df, hue='Duration', style='Duration')
plt.title("Pulse vs Calories (Colored by Duration)")
plt.xlabel("Pulse")
plt.ylabel("Calories")
plt.grid(True)
plt.show()
```

hue='Duration'

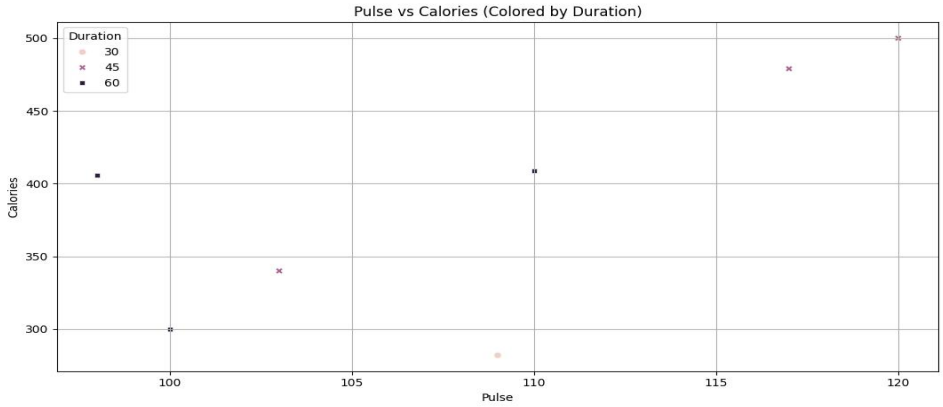
This tells Seaborn to color the points based on the Duration column.

Each unique value in Duration will get a different color.

style='Duration'

This tells Seaborn to use different marker shapes based on the Duration column.

Each unique Duration gets a different symbol (like circle, square, triangle, etc.).



8. Critically evaluate the use of Seaborn vs. Matplotlib in statistical data visualizations.

Matplotlib is the foundational plotting library in Python, while Seaborn is a high-level library built on top of it. The choice between them depends on the specific goals of the visualization: customization vs. statistical insight and speed.

Matplotlib:

Role: The "engine" of Python visualization. It provides a set of low-level "primitives" (lines, points, text, etc.) that can be assembled to create any plot imaginable.

Strengths:

Full Control: It gives you granular control over every single element of a plot—from axis spines and tick marks to annotation placement.

Versatility: Because it's a low-level library, it's not opinionated about what you should plot. It can be used for financial charts, anatomical diagrams, engineering plots, and more.

Weaknesses:

Verbosity: Creating a complex, publication-quality plot often requires many lines of code.

Defaults: Its default styles and color palettes are considered by many to be visually dated.

Data Format: It doesn't work as seamlessly with Pandas DataFrames. You often have to pass Series (columns) manually (e.g., plt.plot(df['x'], df['y'])).

Seaborn:

Role: A "wrapper" for Matplotlib that is specifically designed for statistical data visualization.

Strengths:

Concise Syntax: It can create sophisticated statistical plots like violin plots, pair plots, and regression plots in a single line of code.

Pandas Integration: It is designed to work directly with Pandas DataFrames. You can pass the entire DataFrame and specify column names as strings (e.g., sns.scatterplot(data=df, x='col_x', y='col_y')).

Statistical Awareness: Many of its functions automatically perform statistical estimations (like confidence intervals in a regression plot) as part of the visualization.

Aesthetics: It has visually appealing default themes and color palettes.

Weaknesses:

Less Customization: While you can still access the underlying Matplotlib objects to customize a Seaborn plot, it can sometimes be more cumbersome than starting from scratch in Matplotlib.

Opinionated: It is designed for specific types of statistical plots. If your visualization doesn't fit one of its predefined functions, it can be difficult to create.

Critical Evaluation:

Neither library is inherently "better"; they serve different purposes.

Use Matplotlib when you need to create a highly customized, novel, or non-standard visualization. It's the right tool when you are the "artist" with a precise vision.

Use Seaborn when your primary goal is **exploratory data analysis (EDA)** and statistical inference. It dramatically speeds up the process of understanding relationships and distributions within your data. It's the right tool when you are the "detective" looking for patterns.

A skilled data scientist doesn't choose one over the other; they use both. A common workflow is to use Seaborn for initial exploration and standard plots, and then use Matplotlib to fine-tune the final visualizations for a presentation or publication.

9. Create a complete data analysis pipeline in Python: Load data → Clean → Analyze → Visualize (use `tips.csv` or any dataset you prefer).

This example demonstrates a complete mini-project pipeline using Pandas for analysis and Seaborn/Matplotlib for visualization.

Prefered dataset: exercise data

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# 1. Load the Exercise Dataset
data = {
   'Duration': [60, 45, 45, 30, 60, 60, 45, None],
   'Date': ['2020-12-01', '2020-12-02', '2020-12-03', '2020-12-04',
            '2020-12-05', '2020-12-06', '2020-12-07', '2020-12-08'],
   'Pulse': [110, 117, 103, 109, 98, 100, 120, 115],
   'Maxpulse': [130, 145, 135, 175, 120, 132, 140, 150],
   'Calories': [409, 479, 340, 282, 406, None, 500, 390]
}
df = pd.DataFrame(data)
# 2. Clean the Data
# Check for missing values
print("Missing values:\n", df.isnull().sum())
# Drop rows with missing values
df_clean = df.dropna()
# Convert 'Date' to datetime
df_clean['Date'] = pd.to_datetime(df_clean['Date'])
# 3. Analyze the Data
```

```
print("\nBasic Statistics:")
print(df_clean.describe())
# Correlation between features
print("\nCorrelation Matrix:")
print(df_clean.corr(numeric_only=True))
#group the data by date and calculate the average Pulse for each day.
print("\ngrouping the data:")
daily_avg=df.groupby('Date')['Pulse'].mean()
print(daily_avg)
# 4. Visualize the Data
# Scatter plot: Pulse vs Calories
sns.scatterplot(data=df_clean, x='Pulse', y='Calories', hue='Duration', style='Duration')
plt.title("Pulse vs Calories Burned")
plt.xlabel("Pulse")
plt.ylabel("Calories")
plt.grid(True)
plt.show()
# Line plot: Date vs Duration
sns.lineplot(data=df_clean, x='Date', y='Duration', marker='o')
plt.title("Duration Over Time")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Histogram: Maxpulse distribution
sns.histplot(data=df_clean, x='Maxpulse', bins=5, kde=True, color='skyblue')
plt.title("Distribution of Maxpulse")
plt.xlabel("Maxpulse")
plt.ylabel("Frequency")
plt.show()
```

The above program perfectly follows the pipeline: it loads the data, confirms it's clean, performs two different analyses (groupby and corr), and finally creates three distinct, well-labeled visualizations to clearly present the findings.